biorag

Development History and Design Rationale

Retrieval-augmented QA over PubMed literature
on allosteric kinase inhibitors

Author Florian Ebner
Repository https://github.com/FlorianEbner96/biorag
Licence MIT
Compiled September 2, 2026

This document records *how* the system was built and, more importantly, *why* each layer was designed the way it was. It is deliberately not a feature list: a feature list says what exists, and the interesting content of an engineering project is the set of alternatives that were considered and rejected. Where a decision went against the obvious default, the reasoning is given in full, including the cases where the measured result contradicted the expectation.

Contents

1 The problem and the core promise **1**

2 Timeline **1**

3 Architecture **2**

4 Design decisions **2**
4.1 Abstracts are not chunked by default . . . 2
4.2 Unicode normalised, never ASCII-folded . . . 3
4.3 Exact search, not a vector database . . . 3
4.4 Hybrid retrieval fuses ranks, not scores . . . 3
4.5 Fuzzy title deduplication tuned tight, at 0.97 . . . 3
4.6 Citations are verified, not trusted . . . 4
4.7 Empty retrieval refuses deterministically . . . 4
4.8 Everything is testable offline . . . 4

5 Evaluation **4**
5.1 The gold set . . . 4
5.2 Retrieval results . . . 5
5.3 The result that contradicted the expectation . . . 5
5.4 Retrieval and generation are scored separately . . . 5

6 The HTTP layer **5**

7 Known limitations and next steps **6**

1 The problem and the core promise

A language model asked about allosteric kinase inhibitors will answer from its parametric memory. In a scientific domain this is not merely imprecise; it is unverifiable, which is worse. The reader cannot tell a correct recollection from a fluent fabrication, because both look identical.

Core promise

Every generated claim traces to a specific abstract via its PMID, and *every citation is checked against what was actually retrieved*. Provenance is carried through every layer rather than reconstructed at the end, because provenance rebuilt after the fact is provenance that is wrong sometimes.

The scope was narrowed deliberately to one target class — allosteric kinase inhibitors — rather than oncology or pharmacology at large. A corpus of a few hundred abstracts is large enough to be a genuine retrieval problem and small enough to finish, evaluate and understand alone.

2 Timeline

The system was built in four commits across three days, then extended with an HTTP layer. Each stage is listed with what it added and what question it answered.

Date	Commit	What it added
2026-07-19	`67c34f0`	**Foundation: data through retrieval.** Config, canonical schema, PubMed fetcher (NCBI E-utilities), corpus cleaner, chunker, embedder, vector store, and three retrievers (dense, BM25, hybrid). Synthetic corpus generator added at the same time, so the cleaner could be asserted against known ground truth from the first commit rather than retrofitted with tests later.
2026-07-19	`664c654`	**Generation and evaluation.** Prompt templates, the `Answerer` with citation verification, the gold-set loader and the retrieval metrics. MIT licence added. Dependencies corrected in the same commit: `numpy` and `requests` were used everywhere but never declared (a fresh install relied on them arriving transitively via `torch`), while `anthropic`, `fastapi`, `uvicorn` and `pydantic` were declared but unused and became optional extras. `regex` was listed and never used at all.
2026-07-21	`9cfd6eb`	**Query CLI and batching fix.** `examples/ask.py`: one-shot or interactive querying against the real index, with optional JSON output. Degrades to retrieval-only without an API key.
2026-07-21	`b39e488`	**Gold standard and evaluation runner.** 18 hand-written questions with known relevant PMIDs, and `examples/evaluate.py` to score the three retrievers against them.
2026-09-02	*(this work)*	**HTTP API.** `/health`, `/ask` and `/search` on FastAPI, with Pydantic request and response models, dependency injection for testability, and 17 additional offline tests.

Current size. 2 571 lines of source across 14 modules, 912 lines of tests, 119 tests passing — none requiring network access, an API key, or a model download.

3 Architecture

Data flows in one direction, and every stage preserves the PMID.

Layer	Module	Responsibility
Data — fetch	`data/fetch_pubmed.py`	ESearch → EFetch, rate-limited, cached as raw JSON
Data — clean	`data/clean_corpus.py`	Dedup, boilerplate stripping, retraction and quality filters
Ingestion — chunk	`ingestion/chunker.py`	Section-aware splitting, whole abstracts by default
Ingestion — embed	`ingestion/embedder.py`	Local sentence-transformers, normalised vectors
Ingestion — store	`ingestion/store.py`	Exact NumPy search; optional Chroma backend
Retrieval	`retrieval/retriever.py`	Dense, BM25, and rank-fused hybrid
Generation	`generation/`	Prompting, answering, citation verification
Evaluation	`evaluation/`	Gold set, retrieval and refusal metrics
API	`api/`	FastAPI service over the same components

4 Design decisions

4.1 Abstracts are not chunked by default

The reflexive default in most RAG tutorials is a 200-token sliding window. That convention is inherited from chunking long PDFs and is actively harmful here.

Decision

Keep whole abstracts intact below 2 000 characters (roughly 95% of them). Above that, split only on the author's own section boundaries.

A PubMed abstract is around 250 words and is already a coherent retrieval unit, written by an expert to summarise one paper. Splitting it separates a finding from the condition that qualifies it, and embeds fragments no model was trained on.

4.2 Unicode normalised, never ASCII-folded

Decision

NFKC normalisation only. The idiom `normalize('NFKD').encode('ascii','ignore')` is explicitly rejected.

PI3Kα and PI3Kβ are different proteins with different inhibitors. The standard cleaning idiom collapses both to "PI3K" and destroys the exact distinction a question may turn on.

4.3 Exact search, not a vector database

Decision

Brute-force NumPy dot product as the default store. A Chroma backend exists behind the same interface for persistence.

At this scale a 3000×384 float32 matrix is 4.6 MB and one matrix–vector product takes single-digit milliseconds. Approximate-nearest-neighbour indexes trade recall for speed at 10^5–10^8 vectors; at 10^3 they add a dependency and lose recall in exchange for imperceptible gain.

4.4 Hybrid retrieval fuses ranks, not scores

Decision

Reciprocal Rank Fusion over the two ranked lists.

Cosine similarities and BM25 scores are not commensurable, and BM25's scale is corpus-dependent. Fusing ranks needs no score normalisation and no tuning constant that silently depends on the corpus.

4.5 Fuzzy title deduplication tuned tight, at 0.97

Decision

0.97, not the more intuitive 0.90–0.92.

A group publishing serially on related compounds produces titles sharing more than 90% of their characters while describing entirely different work. Testing against a synthetic corpus with known ground truth showed that a 0.92 threshold silently discarded roughly 15% of genuinely distinct papers. On the real corpus, 0.97 removed 4 near-duplicates out of 413 fetched.

4.6 Citations are verified, not trusted

Decision

The prompt requests inline `[PMID]` markers; the answerer then checks every cited PMID against what was actually in the context. `Answer.is_grounded` becomes false when a citation was fabricated.

This is the single most damaging failure mode the system has, precisely because the output looks entirely credible. A model citing a plausible PMID that was never retrieved produces text that a reader cannot distinguish from a correct answer.

4.7 Empty retrieval refuses deterministically

If nothing was retrieved, the system returns a refusal without calling the model at all. Hoping a model refuses on empty context is a coin flip; refusing in code is free and certain.

4.8 Everything is testable offline

Decision

`data/synthetic.py` generates PubMed-shaped records with deliberately planted pathologies — retractions, boilerplate, duplicate titles, one-line abstracts. A hashing `FakeEmbedder` and a scripted `FakeLLM` stand in for the model and the API.

Tests that need the network get skipped, and skipped tests protect nothing. All 119 tests run with no network, no API key and no model download.

5 Evaluation

5.1 The gold set

18 hand-written questions against the real corpus, written *before* tuning anything — questions written after seeing what the system does well measure the author's selection bias rather than the system.

Question type	Count
Identifier (specific compound codes)	5
Factual	4
Conceptual	4
Unanswerable (expects refusal)	**4**
Multi-hop	1
Total	18

Why 22% of the gold set is deliberately unanswerable

This is the category most evaluation sets omit, and the only one that detects a system which answers everything confidently. A RAG system scoring perfectly on factual questions while never refusing is useless in a scientific setting, and `refusal_recall` cannot be computed at all without such questions.

5.2 Retrieval results

Corpus: 409 PubMed abstracts, 494 chunks. Scored over the 14 answerable questions at $k = 5$.

Retriever	recall@5	precision@5	MRR	hit rate
Dense (semantic, MiniLM)	0.274	0.100	0.339	0.500
BM25 (keyword baseline)	0.405	0.143	**0.393**	0.571
Hybrid (RRF)	**0.429**	**0.157**	0.392	**0.643**

5.3 The result that contradicted the expectation

The textbook expectation is that semantic retrieval beats keyword search. Here it did not: dense retrieval was the *weakest* of the three. Two honest limitations explain it, and neither was known in advance.

1. **The gold set leans on exact identifiers.** Nine of the 14 answerable questions hinge on a specific compound code — EAI045, ABL001, MK-2206, IACS-13909, SHP099. This is precisely what BM25 is built for and what a general embedding model handles poorly, since a compound code is a rare token it has barely seen.
2. **The embedding model is general-purpose, not biomedical.** `all-MiniLM-L6-v2` was never trained on this vocabulary. A domain model such as `S-PubMedBert-MS-MARCO` should lift the dense numbers materially — and the evaluation harness exists precisely to *measure* that swap rather than assume it.

The conclusion that does hold is the one that matters: **fusing the two beats either alone** on recall and hit rate. Neither retriever is redundant. BM25 nails the identifiers; dense contributes on the conceptual questions whose wording shares no tokens with the abstract that answers them.

A note on BM25 and negative IDF

On a small synthetic corpus the domain terms (“allosteric”, “kinase”, “inhibitor”) appear in more than half of all documents and take negative IDF, so BM25 scores come out negative — correctly reporting that the domain’s defining terms carry no discriminative power. On the real 409-abstract corpus they do not, which is itself an argument for evaluating on real rather than toy data.

5.4 Retrieval and generation are scored separately

In that order, and on purpose. If recall@5 is 0.4, no amount of prompt engineering will fix the answers, because the relevant abstract was never in the context. Debugging generation when the fault is upstream in retrieval is the most common time sink in these projects.

6 The HTTP layer

The API exposes the same components over HTTP without changing their contracts.

Endpoint	Method	Purpose
`/health`	GET	Readiness, not just liveness — reports indexed chunk count and whether generation is configured
`/ask`	POST	Cited answer when a key is set, otherwise the retrieved abstracts
`/search`	GET	Retrieval only, never generation — the guaranteed-free path

Load once, in a lifespan handler

At import time the module would be unimportable without a built index, which breaks the test suite. Per request it would pay an ≈80 MB model load on every call. FastAPI’s `lifespan` does it exactly once and is the documented replacement for the deprecated `@app.on_event("startup")`.

Dependencies are injected, never constructed in a handler

The rest of the codebase already works this way — `Answerer` takes an injected LLM client, `RAGPipeline` an injected retriever — and that is exactly why the tests need no key, no network and no model. Handlers depend on `get_deps()`, which tests override with fake-backed instances, so the API layer does not become the one place that breaks the property.

A missing index returns 503, not 500

“I have no index” is not a server error, it is *not ready to serve*. An orchestrator keeps a 503 container out of rotation until its volume is mounted; a 500 looks like a crash loop. Correspondingly, `/health` reports the chunk count, because the failure that actually happens in a container is not a dead process but a live one whose index volume was never mounted.

The verification result is part of the response. `is_grounded` and `hallucinated_citations` are first-class response fields rather than server-side log lines. An API returning the answer text while dropping the verification would hand back exactly the unverifiable output this project exists to avoid.

7 Known limitations and next steps

Stated plainly, because a portfolio project that claims no limitations is not credible.

- **The embedding model is not biomedical.** This is the single clearest improvement available, and the harness exists to measure it.
- **Absolute retrieval numbers are modest.** recall@5 of 0.43 on an identifier-heavy gold set is a starting point, not a finished result.
- **The gold set is small** at 18 questions, and written by one person. It detects gross regressions, not subtle ones.
- **Groundedness is checked at citation level, not claim level.** A model can cite a genuinely retrieved PMID for a claim that abstract does not make. Claim-level verification is the natural next layer.
- **No CI yet.** The tests run offline in under two seconds, so wiring them into GitHub Actions is the obvious next step and would make the test suite a gate rather than a habit.
- **Docker** is the remaining packaging step, so the whole system runs with one command.

Source and full documentation: `https://github.com/FlorianEbner96/biorag`